Как выбирать типы данных в PostgreSQL

Встроенные и пользовательские типы, паттерны использования

Владимир Добрышкин

20.10.2026

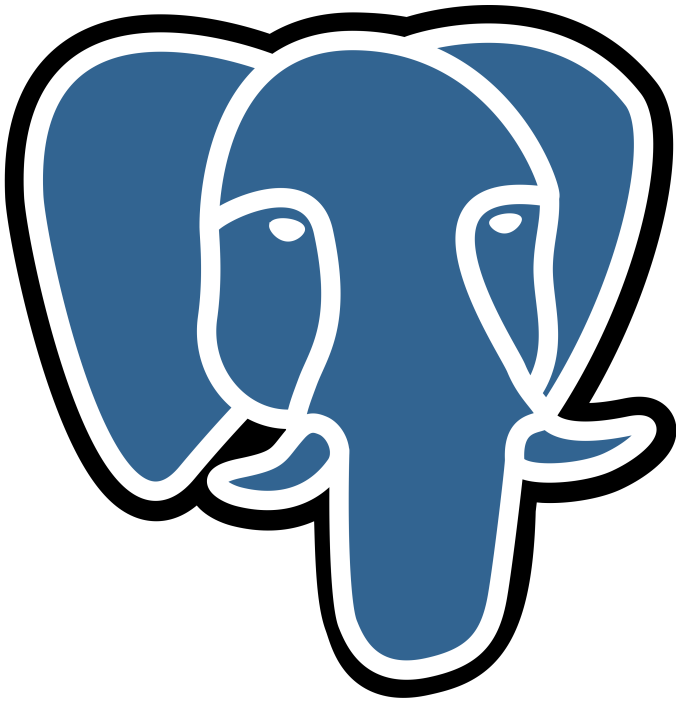
ИТМО

Мотивация

Улучшить навык моделирования предметной области, осознанно выбирая типы данных.

План доклада

1. Пользовательские типы данных.
2. Паттерны использования различных типов данных.



ИТМО

1

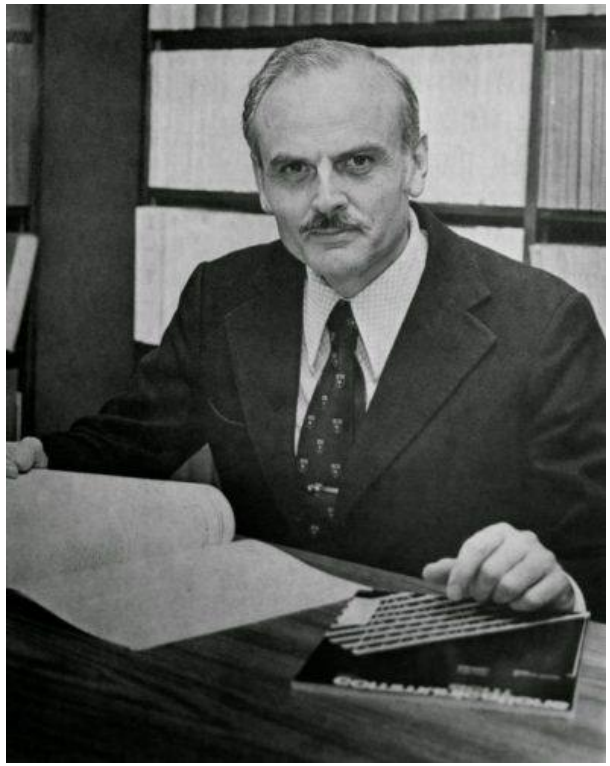
Пользовательские типы данных

Модель данных

Классическое определение было дано Эдгаром Коддом (*Edgar F. Codd*) в 1980 году в статье *Data Models in Database Management*.

Комбинация трёх компонентов:

1. Набор типов структур данных.
2. Набор операций, которые можно применять к экземплярам этих типов данных.
3. Набор общих правил целостности.



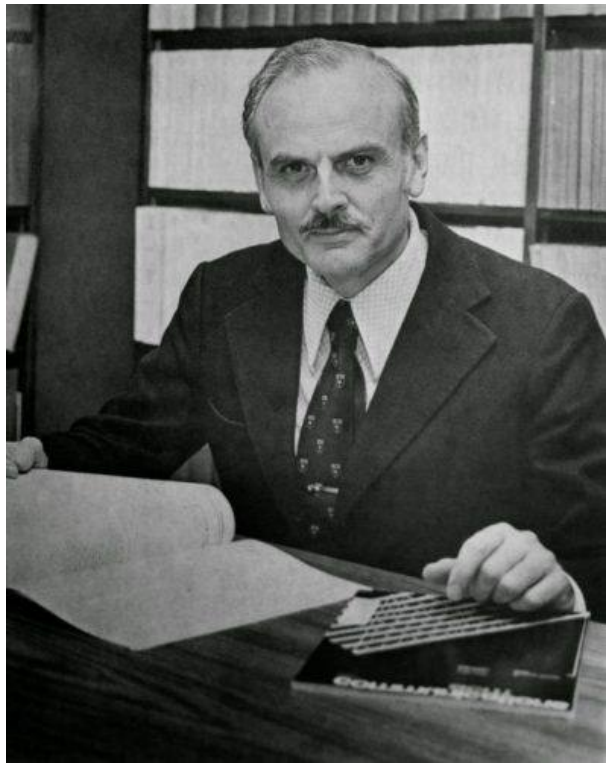
Эдгар Кодд (1923-2003), автор реляционной модели данных

Модель данных

Классическое определение было дано Эдгаром Коддом (*Edgar F. Codd*) в 1980 году в статье *Data Models in Database Management*.

Комбинация трёх компонентов:

1. **Набор типов структур данных.**
2. Набор операций, которые можно применять к экземплярам этих типов данных
3. Набор общих правил целостности.



Эдгар Кодд (1923-2003), автор реляционной модели данных

Встроенные типы данных

- Целочисленные типы данных: `SMALLINT`, `INTEGER`, `BIGINT`.
- Логический тип данных: `BOOLEAN`.
- Числа с плавающей запятой: `REAL`, `DOUBLE PRECISION`.
- Символьные строки: `CHAR`, `VARCHAR`, `TEXT`.
- Дата и время: `DATE`, `TIME`, `TIMESTAMP`, `TIMESTAMP WITH TIME ZONE`.
- Куча других типов данных, включая массивы, JSON, XML, UUID и т.д.

ИТМО

Зачем тогда нужны пользовательские типы данных?

Предметная область: таск-трекер

Таблица задач

```sql
CREATE TABLE tasks (
    task_id BIGSERIAL PRIMARY KEY,
    title TEXT NOT NULL,
    priority TEXT NOT NULL -- низкий, средний, высокий
);

INSERT INTO tasks (title, priority) VALUES
    ('Выучить теорию на лабу', 'low'),
    ('Сдать первую лабу', 'medium'),
    ('Подготовиться к экзамену', 'high');
```

Нет проверки допустимых значений

Вставка “левых” значений

```
INSERT INTO tasks (title, priority) VALUES
    ('Начать делать вторую лабу', 'meedium'),
    ('Выучить теорию на вторую лабу', 'extra-high'),
    ('Сходить на английский', 'LOW')
```

Одно из возможных решений

Использование CHECK

```
CREATE TABLE tasks (
    task_id BIGSERIAL PRIMARY KEY,
    title TEXT NOT NULL,
    priority TEXT NOT NULL CHECK (priority IN ('low',
'medium', 'high'))
);
```

Проблема 1: одна проверка в трёх таблицах

Задачи

```
CREATE TABLE tasks (
  id BIGSERIAL PRIMARY KEY,
  title TEXT NOT NULL,
  priority TEXT NOT NULL
    CHECK (priority IN (
      'low', 'medium',
      'high'
    ))
);
```

Цели

```
CREATE TABLE goals (
  id BIGSERIAL PRIMARY KEY,
  title TEXT NOT NULL,
  target_date DATE,
  priority TEXT NOT NULL
    CHECK (priority IN (
      'low', 'medium',
      'high'
    ))
);
```

События

```
CREATE TABLE events (
  id BIGSERIAL PRIMARY KEY,
  title TEXT NOT NULL,
  starts_at TIMESTAMPTZ,
  location TEXT,
  priority TEXT NOT NULL
    CHECK (priority IN (
      'low', 'medium',
      'high'
    ))
);
```

Проблема 2: добавление значения в трёх местах

Задачи

```
ALTER TABLE tasks
  DROP CONSTRAINT
    tasks_priority_check,
  ADD CHECK (
    priority IN (
      'low', 'medium',
      'high', 'urgent'
    )
  );
```

Цели

```
ALTER TABLE goals
  DROP CONSTRAINT
    goals_priority_check,
  ADD CHECK (
    priority IN (
      'low', 'medium',
      'high', 'urgent'
    )
  );
```

События

```
ALTER TABLE events
  DROP CONSTRAINT
    events_priority_check,
  ADD CHECK (
    priority IN (
      'low', 'medium',
      'high', 'urgent'
    )
  );
```

Проблема 3: CHECK не задаёт порядок приоритетов

Сортировка по правилам строк

```
SELECT title, priority
FROM tasks
ORDER BY priority DESC;

-- Названия приоритетов
-- сравниваются как строки.
```

Порядок важности задаём вручную

```
SELECT title, priority
FROM tasks
ORDER BY CASE priority
  WHEN 'urgent' THEN 4
  WHEN 'high'   THEN 3
  WHEN 'medium' THEN 2
  WHEN 'low'    THEN 1
END DESC;
```

ИТМО

Решение: пользовательский тип данных с помощью ENUM

ENUM

ENUM – тип данных с заранее заданным набором допустимых значений.

Поддержка enum есть в большинстве языков программирования:

Python

```python
from enum import Enum

class Priority(Enum):
  LOW = "low"
  MEDIUM = "medium"
  HIGH = "high"
```

Java

```java
enum Priority {
  LOW,
  MEDIUM,
  HIGH
}
```

C++

```cpp
enum class Priority {
  LOW,
  MEDIUM,
  HIGH
};
```

Пример использования ENUM

Сущность tasks

```
CREATE TYPE PRIORITY AS ENUM (
    'low',
    'medium',
    'high'
);

CREATE TABLE tasks (
    task_id BIGSERIAL PRIMARY KEY,
    title TEXT NOT NULL,
    task_priority PRIORITY NOT NULL
);
```

Добавляем значение в одном месте

Изменяем общий тип

```
ALTER TYPE PRIORITY
  ADD VALUE 'urgent'
  AFTER 'high';
```

Значение доступно во всех таблицах

```
INSERT INTO tasks(title, task_priority) VALUES
    ('Задача', 'urgent');

INSERT INTO goals(title, goal_priority) VALUES
    ('Цель', 'urgent');

INSERT INTO events(title, event_priority) VALUES
    ('Событие', 'urgent');
```

Сортировка по порядку значений ENUM

Порядок значений типа

```
CREATE TYPE PRIORITY AS ENUM (
  'low',
  'medium',
  'high',
  'urgent'
);
```

Запрос

```
SELECT task_priority
FROM tasks
ORDER BY
  task_priority DESC;
```

Результат

```
task_priority
-------------
urgent
high
medium
low
```

Почтовый индекс через CHECK

Таблица адресов

```
CREATE TABLE addresses (
  id BIGSERIAL PRIMARY KEY,
  city TEXT NOT NULL,
  postal_code TEXT NOT NULL CHECK (postal_code ~ '^[0-9]{6}$')
);
```

DOMAIN

DOMAIN – пользовательский тип на основе существующего типа, которому можно задать ограничения на допустимые значения.

Создаём тип для почтового индекса

```
CREATE DOMAIN POSTAL_CODE AS TEXT
CHECK (VALUE ~ '^[0-9]{6}$');
```

Используем в таблице адресов

```
CREATE TABLE addresses (
  id BIGSERIAL PRIMARY KEY,
  city TEXT NOT NULL,
  postal_code POSTAL_CODE NOT NULL
);
```

Что решает DOMAIN

- Даём понятию отдельное имя, соответствующее предметной области.
- Проверку формата описываем в типе, а не в каждом столбце отдельно.
- Можно переиспользовать один тип в нескольких таблицах.
- Сохраняем обычные операции исходного типа TEXT.

COMPOSITE

COMPOSITE – составной тип, который объединяет несколько именованных полей в одно значение.

- У каждого поля есть свой тип.
- Можно использовать уже созданные пользовательские типы.
- Можно обращаться к отдельным полям составного значения.

Пример: адрес

```
CREATE TYPE ADDRESS AS (
    city TEXT,
    street TEXT,
    postal_code POSTAL_CODE
);

-- Санкт-Петербург
SELECT (ROW('Санкт-Петербург', 'Кроверкский
пр-т', '197198')::ADDRESS).city;
```

Диапазоны значений

- **RANGE** — один диапазон значений. Например, время работы без перерыва.
- **MULTIRANGE** — несколько непересекающихся диапазонов. Например, время работы с перерывом.

Пример: время работы

```
CREATE TYPE TIME_RANGE AS RANGE (
  SUBTYPE = TIME
);

SELECT '[09:00,18:00)'::TIME_RANGE;

SELECT '{[09:00,13:00),
[14:00,18:00)}'::TIME_MULTIRANGE;
```

Какие пользовательские типы использовать?

> “It’s all about figuring out what’s going to work best in a given situation.”
>
> — Grant Fritchey, Simple Talk, 2026

ИТМО

2

Паттерны применения типов данных

Храните даты событий вместо относительного времени

Возраст человека, стаж работы и другие временные параметры меняются с течением времени. Храним дату события, а относительные значения вычисляем при необходимости.

Для календарной даты используем DATE. Если нужно сохранить конкретный момент времени с учётом часовых поясов, используем TIMESTAMPTZ.

Не храним текущий возраст

```
CREATE TABLE staff (
  id BIGSERIAL PRIMARY KEY,
  name TEXT NOT NULL,
  age INTEGER NOT NULL
);
```

Храним дату рождения

```
CREATE TABLE staff (
  id BIGSERIAL PRIMARY KEY,
  name TEXT NOT NULL,
  birth_date DATE NOT NULL
);
```

Используйте VARCHAR(n) вместо CHAR(n)

- CHAR(n) дополняет короткие строки пробелами до заданной длины. Эти пробелы занимают место, поэтому хранение коротких значений получается менее компактным.
- VARCHAR(n) сохраняет исходную строку, и т.к. CHAR(n) не даёт никаких преимуществ, то используем VARCHAR(n).

CHAR(n)

```
SELECT octet_length(
  'abc'::CHAR(10)
);
-- 10 байт
```

VARCHAR(n)

```
SELECT octet_length(
  'abc'::VARCHAR(10)
);
-- 3 байта
```

Не храните номер телефона числом

Номер телефона — это обозначение, а не числовая величина. Над ним не выполняют арифметические операции.

Числовой тип не сохраняет начальные нули и знак “+”. Поэтому номер телефона храним строкой или DOMAIN.

Не храним телефон числом

```
CREATE TABLE staff (
  id BIGSERIAL PRIMARY KEY,
  name TEXT NOT NULL,
  phone BIGINT NOT NULL
);
```

Храним телефон строкой

```
CREATE TABLE staff (
  id BIGSERIAL PRIMARY KEY,
  name TEXT NOT NULL,
  phone TEXT NOT NULL
);
```

Представляйте длительность с помощью INTERVAL

- TIME хранит время суток, а не длительность. Поэтому значение ‘02:00:00’ допустимо, а ‘30:00:00’ будет отклонено.
- Для длительности используем INTERVAL. Он позволяет хранить продолжительность больше суток и выполнять операции с временными интервалами.

TIME

```
SELECT TIME '02:00:00';
-- Допустимое время суток

SELECT TIME '30:00:00';
-- Ошибка
```

INTERVAL

```
SELECT INTERVAL '2 hours';
-- 02:00:00

SELECT INTERVAL '30 hours';
-- 30:00:00
```

Не храните строками то, что не является строкой

- Тип данных задаёт допустимые значения и доступные операции. Если записать дату в TEXT, база будет работать с ней как со строкой.
- DATE проверяет существование даты и позволяет выполнять операции с ней. Например, прибавить день без предварительного преобразования строки.

Строковое представление

```
SELECT '2026-02-30'::TEXT;
-- Несуществующая дата принимается

SELECT '2026-02-01'::TEXT + 1;
-- Ошибка
```

Подходящий тип данных

```
SELECT DATE '2026-02-30';
-- Несуществующая дата отклоняется

SELECT DATE '2026-02-01' + 1;
-- 2026-02-02
```

Ограничивайте длину строковых значений

- Слишком длинный пользовательский ввод может приводить к лишнему расходу памяти, места и времени обработки.
- Используем VARCHAR(n), чтобы закрепить ограничение в базе данных.

Без ограничения длины

```
CREATE TABLE comments (
  id BIGSERIAL PRIMARY KEY,
  author_name TEXT NOT NULL,
  body TEXT NOT NULL
);
```

Ограничиваем длину

```
CREATE TABLE comments (
  id BIGSERIAL PRIMARY KEY,
  author_name VARCHAR(100) NOT NULL,
  body VARCHAR(2000) NOT NULL
);
```

Представляйте денежные суммы с помощью NUMERIC

- MONEY зависит от денежной локали. Например при российской локали суммы отображаются в рублях. Сам тип не сохраняет валюту, а смена локали не выполняет конвертацию.
- REAL и DOUBLE PRECISION хранят десятичные значения приближённо. Для денежных расчётов, требующих точности, используем NUMERIC.

MONEY

```
CREATE TABLE payments (
  id BIGSERIAL PRIMARY KEY,
  payer_name TEXT NOT NULL,
  amount MONEY NOT NULL
);
```

NUMERIC

```
CREATE TABLE payments (
  id BIGSERIAL PRIMARY KEY,
  payer_name TEXT NOT NULL,
  amount NUMERIC(12, 2) NOT NULL
);
```

ИТМО

Спасибо за внимание!

Вопросы?

t.me/vodobryshkin github.com/vodobryshkin

Источники

- E. F. Codd. Data Models in Database Management. 1980.
- Michael Stonebraker. Object-Relational DBMS — The Next Wave.
- Документация PostgreSQL 18. Глава 8. Data Types.
- Grant Fritchey. How User-Defined Types work in PostgreSQL. Simple Talk.
- PostgreSQL Wiki. Don’t Do This.
- Laurenz Albe. Don’t do that!. PGConf.EU 2025. Доклад и слайды.